

TULens: A Multilingual RAG Assistant for Navigating TU Wien's Documentation

Group 17

1. Abstract

TULens addresses the challenge prospective and current students face when navigating TU Wien's fragmented documentation ecosystem. Information about admission requirements, curricula, and administrative procedures is scattered across websites, PDFs, and portals (often in German) making it difficult to locate and synthesize. We developed a Retrieval-Augmented Generation (RAG) system that ingests heterogeneous sources (HTML pages, curriculum PDFs), stores them in a vector database, and generates coherent, cited answers to natural-language queries. The key technical challenge was achieving accurate retrieval across multilingual, multi-format documents while ensuring factually grounded responses.

2. Description

2.1 User Journey

A student opens the TULens web interface and is greeted by a chat interface with a sidebar showing system status and document statistics. The student types their first question: *"What are the admission requirements for the Data Science master program?"* The system retrieves relevant chunks from scraped informatics pages, admission guidelines, and curriculum PDFs. The student receives a synthesized answer in English with expandable source citations linking to original documents.

The experience is defined by three key features:

- **Multi-Turn Conversations:** Users can ask follow-ups like *"What about for non-EU applicants?"* without restating context, as the system remembers the chat history.
- **PDF Upload:** Students can drop in personal documents (e.g., curriculum plans) for real-time, session-specific analysis.
- **Source Transparency:** Every response includes expandable citations to official documents.

2.2 Technical Approach

We built a pipeline consisting of:

1. **Web Scraping:** Using Scrapy and LangChain's RecursiveUrlLoader.
2. **PDF Extraction:** Via PyPDFLoader with custom text cleaning.
3. **Document Chunking:** 800 tokens with 150 overlap using RecursiveCharacterTextSplitter.
4. **Embedding:** Using BAAI/bge-small-en-v1.5 for multilingual support.
5. **Vector Storage:** Stored in ChromaDB.
6. **Generation:** Using Llama-3.3-70B via Groq API.

We built an automated web crawler that systematically collects information from TU Wien's websites. The crawler navigates through program pages and admission sections while skipping irrelevant content like news and events. It automatically discovers and downloads linked PDF documents such as curriculum guides. The system preserves context (like program association and site hierarchy) to facilitate citation. The crawler converts web pages into clean, structured text, stripping away navigation menus and clutter while keeping meaningful content intact.

Key design choices include:

- **MMR Retrieval:** Balances relevance and diversity to prevent redundant chunks.
- **Metadata Enrichment:** Chunks carry source URL, program, and content type for filtered retrieval.
- **Dual Vector Stores:** Combines persistent pre-scraped data (Docker-hosted) with ephemeral user uploads.
- **Prompt Engineering:** Enforces English responses, citations, and acknowledgment of knowledge gaps.

2.3 Challenges Faced

1. **PDF Text Quality:** Curriculum PDFs contained broken formatting. We developed regex-based cleaning pipelines to normalize text.
2. **Multilingual Retrieval:** BGE embeddings handled cross-lingual matching, though specific German administrative terms occasionally caused misses.
3. **Context Window Limits:** We empirically tuned chunk size (800) and retrieval count (k=6) to balance context limits with answer completeness.
4. **Hallucination Control:** Early unsourced claims were mitigated by adding explicit grounding instructions and citation requirements.

2.4 Demo on YouTube

You can watch the demo of our project right here: [Demo Link](#)

3. Evaluation

The TULens system was evaluated using a manually curated set of factual questions related to studies, derived from the same domain as the scraped documentation. The evaluation focused on the factual correctness and completeness of generated answers.

3.1 Evaluation Methodology

The evaluation dataset included admission-related questions (applications, enrollment, and other requirements) and curriculum-related questions (programs, course details, faculty offerings). Answers were manually rated on a 1–5 scale. This grading scheme allows a more nuanced evaluation than binary correctness and better reflects real-world usefulness.

3.2 Evaluation Results

Figure 1 shows that 85% of admission-related questions were completely or mostly correctly answered. Figure 2 shows that curriculum-related questions have even higher accuracy, with 87% answered mostly or completely correct. Errors mainly occurred with date-specific deadlines and EU vs. non-EU admission rules, which are sensitive to wording and frequent updates, making static retrieval difficult.

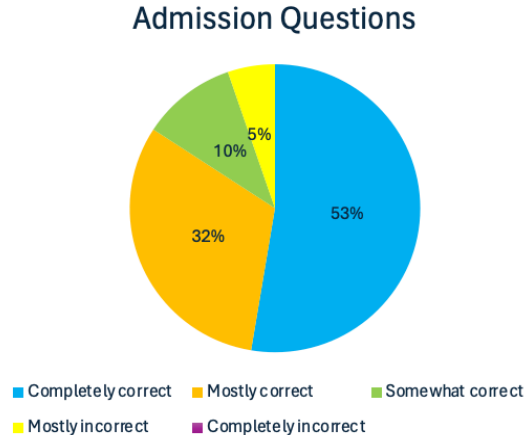


Figure 1: Evaluation results for admission-related questions.

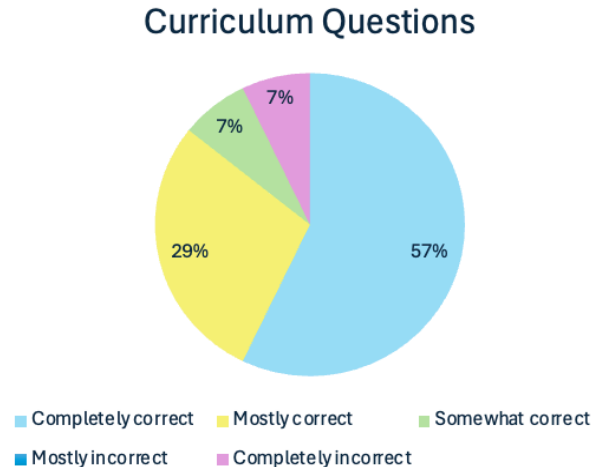


Figure 2: Evaluation results for curriculum-related questions.

Overall, 55% of all answers were evaluated as completely correct, and 84.85% (completely or mostly correct) were considered largely accurate and useful. Remaining limitations are primarily related to time-sensitive administrative content, which could be addressed through more frequent re-scraping or integration with live data sources.

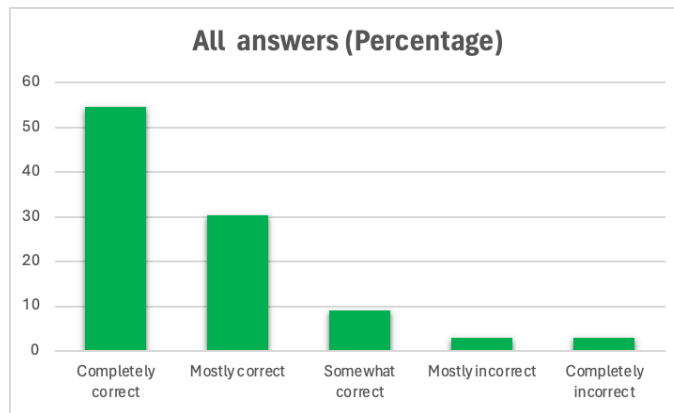


Figure 3: Overall evaluation results across all questions.

4. Reflection

4.1 Does the system increase or decrease human agency?

Increases agency. TULens empowers students to independently access documentation without relying on staff or German proficiency. By providing citations, users verify information and make informed decisions. The system augments human judgment, allowing students to follow official processes with reduced friction.

4.2 Would you develop this further?

Yes. The prototype validates that RAG improves information access. Future work includes:

- Expanding coverage to all faculties.
- Integrating TISS API for real-time course data.
- Adding persistent session storage to multi-turn conversations.
- Implementing user feedback loops.

The modular architecture supports these extensions without fundamental redesign.